

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde  
xer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_58a19e3e-3ce\agent-  
a2db841f506e6f7c4.jsonl`  
- SHA-256: `8a348621f01a9c51dbeaa8ed30aa9ef4a813cd4feb618a224ae6bd7b1d54b69e`  
- Source modified: `2026-07-06T23:21:42+00:00`  
- Imported at: `2026-07-08T16:00:05+00:00`  
- project: `wf_58a19e3e-3ce`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T23:19:54.510Z)

Reviewing GitHub PR #54 "feat(policy): add F5 deep inspection budget boundary" (branch feat/f5-deep-inspection-budget -> main). ADR 0012 F5.
+364/-4, 3 files. Checked-out copy at C:/Users/avidu/Projects/_wt-pr54 (read there or 'git -C C:/Users/avidu/Projects/Telegram show origin/feat/f5-deep-inspection-budget:<path>').
Run repros: PYTHONPATH=C:/Users/avidu/Projects/_wt-pr54/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>.

WHAT F5 DOES (policy.py) ? BOUNDARY ONLY, no downloader/ffprobe/hash/scan (explicitly deferred):

```
- PolicyBudget(frozen dataclass): max_bytes_per_item, timeout_s, max_concurrent, + private  
_semaphore built in __post_init__ (via object.__setattr__)  
  when max_concurrent>0. async inspect(fetcher, content_item): 'async with self._semaphore:  
return await asyncio.wait_for(fetcher.inspect(item,self), timeout=timeout_s)'.  
- ContentFetcher(Protocol): async inspect(content_item, budget)->StageResult.  
PolicyContext.fetcher/budget now typed (were object|None in F1).  
- load_deep_inspection_enabled(policy): deep_inspect bool OR {enabled: bool}; invalid ->  
ValueError -> stage fail-OPEN (pass, invalid_deep_inspection_config).  
- DeepInspectionStage (Tier 4, LAST after topic/url/caption): opt-out if not enabled -> pass.  
_deep_inspection_budget_result checks (in order, all  
  BEFORE calling fetcher): budget present, budget valid (max_bytes/timeout/max_concurrent all  
>0), content_item.size_bytes present, size<=max_bytes ->  
  returns a FAIL StageResult on any violation. Then if fetcher missing -> fail. Else  
budget.inspect(...) under semaphore+wait_for; TimeoutError -> fail 'budget exceeded'.  
- Live wiring (_evaluate_policy_stages) creates PolicyContext WITHOUT fetcher/budget (both  
None), so deep_inspect enabled -> stage fails 'budget missing'  
  -> video DROPPED (fail-closed). This is intentional/dormant (no /set deep_inspect command  
exists).
```

VERIFIED by main reviewer: gate green (543 passed @ 94.57%); budget checks run BEFORE the
fetcher; deep_inspect enabled-without-budget drops the item
(fail-closed, safe direction); default (deep_inspect unset) unaffected. Comprehensive tests
(missing/invalid budget, missing size, size-exceeded, timeout,
concurrency max_active==1, missing fetcher, valid delegate).

Ground rules: READ code, cite file:line. Run repros. Report ONLY defects in THIS PR's diff. No
praise. Concrete failure scenario, ranked.
Severity: high (budget/timeout/concurrency BYPASS ? heavy work runs unbounded, or a real bug),
medium (real latent/design), low (note/hygiene/intentional-fail-closed).

ADVERSARIALLY VERIFY this finding ? try to REFUTE it. Read cited code, run a repro with
PYTHONPATH=C:/Users/avidu/Projects/_wt-pr54/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe.
CONFIRMED only if the defect genuinely ships in THIS PR's diff and produces the described wrong
behaviour. REFUTED if handled / can't occur /

intentional-and-safe / out-of-scope-deferred. PLAUSIBLE if unsettled. Corrected severity (a real budget bypass = high; intentional fail-closed note = low).

FINDING (budget_enforcement): Negative or zero-ish content size_bytes passes the max_bytes budget check and delegates to the fetcher

severity low | src/tg_video_filter/policy.py:745

summary: _deep_inspection_budget_result only rejects size_bytes when it is None or strictly greater than max_bytes_per_item. A nonsensical negative size (from a buggy/hostile provider metadata path) satisfies size_bytes <= max_bytes_per_item and is treated as a valid, in-budget item, delegating to the fetcher.

scenario: A ContentItem arrives with size_bytes=-5 (provider/metadata glitch). The stage treats it as within budget and calls fetcher.inspect, when the safer interpretation of a negative/invalid size is 'size unknown/untrustworthy -> refuse' like the missing-size branch already does.

evidence: Repro: size=-5 with max_bytes=100 -> passed=True, reason='delegated' (delegates to fetcher). The size<=0 case is never rejected; only size_bytes is None (policy.py:737-744) and size_bytes > max (policy.py:745-753) are handled.

rec: Treat size_bytes <= 0 the same as missing size (fail with missing/invalid size), so only a positive, in-range size delegates to the fetcher. Low severity because timeout+semaphore still bound the delegated work; this is a defense-in-depth/correctness nit, not a heavy-work-unbounded bypass.

2. assistant (2026-07-06T23:19:59.275Z)

I'll verify this finding by reading the cited code and running a repro.

3. user (2026-07-06T23:20:00.501Z)

```
710         except TimeoutError:
711             return StageResult.fail(
712                 self.stage_name,
713                 "deep inspection budget exceeded",
714                 budget_exceeded=True,
715                 exceeded_budget="timeout_s",
716                 timeout_s=ctx.budget.timeout_s,
717             )
718
719
720 def _deep_inspection_budget_result(ctx: PolicyContext) -> StageResult | None:
721     budget = ctx.budget
722     if budget is None:
723         return StageResult.fail(
724             DeepInspectionStage.stage_name,
725             "deep inspection budget missing",
726             missing_budget=True,
727         )
728     if budget.max_bytes_per_item <= 0 or budget.timeout_s <= 0 or budget.max_concurrent
729     <= 0:
730         return StageResult.fail(
731             DeepInspectionStage.stage_name,
732             "deep inspection budget invalid",
733             invalid_budget=True,
734             max_bytes_per_item=budget.max_bytes_per_item,
735             timeout_s=budget.timeout_s,
736             max_concurrent=budget.max_concurrent,
737         )
738     size_bytes = ctx.content_item.size_bytes
739     if size_bytes is None:
740         return StageResult.fail(
741             DeepInspectionStage.stage_name,
742             "deep inspection budget cannot be enforced without content size",
743             missing_size_bytes=True,
744             max_bytes_per_item=budget.max_bytes_per_item,
745         )
746     if size_bytes > budget.max_bytes_per_item:
747         return StageResult.fail(
```

```

747         DeepInspectionStage.stage_name,
748         "deep inspection budget exceeded",
749         budget_exceeded=True,
750         exceeded_budget="max_bytes_per_item",
751         size_bytes=size_bytes,
752         max_bytes_per_item=budget.max_bytes_per_item,
753     )
754     return None
755
756
757     async def evaluate_pipeline(
758         ctx: PolicyContext,
759         stages: list[PolicyStage],
760     ) -> tuple[bool, list[StageResult]]:
761         """Evaluate stages in order and short-circuit on the first failure."""
762         results: list[StageResult] = []
763         for stage in stages:
764             result = await stage.evaluate(ctx)
765             results.append(result)
766             if not result.passed:
767                 return False, results
768         return True, results
769
770
771     __all__ = [
772         "CaptionSafetyConfig",
773         "CaptionSafetyStage",
774         "ContentFetcher",
775         "DeepInspectionStage",
776         "DomainReputationConfig",
777         "ExtractedUrl",
778         "PolicyContext",
779         "PolicyBudget",

```

4. assistant (2026-07-06T23:20:02.955Z)

The finding acknowledges the code path. Let me verify the size_bytes semantics and run a repro.

5. user (2026-07-06T23:20:04.009Z)

```

670         metadata["timed_out_caption_patterns"] = list(timed_out_patterns)
671     return metadata
672
673
674     class DeepInspectionStage:
675         """ADR 0012 Tier 4 opt-in boundary for budgeted heavy inspection."""
676
677         stage_name = "deep-inspection"
678
679         async def evaluate(self, ctx: PolicyContext) -> StageResult:
680             try:
681                 enabled = load_deep_inspection_enabled(ctx.policy)
682             except (json.JSONDecodeError, ValueError) as exc:
683                 log.warning(
684                     "deep inspection disabled due to invalid config policy_id=%s",
685                     ctx.policy.id,
686                     exc_info=True,
687                 )
688                 return StageResult.pass_(
689                     self.stage_name,
690                     f"invalid deep inspection config ignored: {exc}",
691                     invalid_deep_inspection_config=True,
692                 )
693             if not enabled:
694                 return StageResult.pass_(self.stage_name, "deep inspection disabled")

```

```

695
696         budget_result = _deep_inspection_budget_result(ctx)
697         if budget_result is not None:
698             return budget_result
699
700         if ctx.fetcher is None:
701             return StageResult.fail(
702                 self.stage_name,
703                 "deep inspection fetcher missing",
704                 missing_fetcher=True,
705             )
706
707         assert ctx.budget is not None
708         try:
709             return await ctx.budget.inspect(ctx.fetcher, ctx.content_item)
710         except TimeoutError:
711             return StageResult.fail(
712                 self.stage_name,
713                 "deep inspection budget exceeded",
714                 budget_exceeded=True,

```

6. assistant (2026-07-06T23:20:05.879Z)

Now let me look at the ContentItem definition to understand size_bytes semantics.

7. user (2026-07-06T23:20:07.060Z)

```

C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\telethon_client.py:97:
size_bytes=media.size,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\telethon_client.py:234:
meta.size_bytes is not None
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\telethon_client.py:242:
meta.size_bytes,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\telethon_client.py:252:
meta.size_bytes,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\telethon_client.py:310:
size_bytes=meta.size_bytes,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\telethon_client.py:348:
size_bytes=meta.size_bytes,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\telethon_client.py:415:
size_bytes=meta.size_bytes,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\telethon_client.py:422:
size_bytes=meta.size_bytes,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\backfill.py:194:
meta.size_bytes is not None
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\backfill.py:202:
meta.size_bytes,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\backfill.py:212:
meta.size_bytes,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\backfill.py:274:
size_bytes=meta.size_bytes,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\backfill.py:295:
size_bytes=meta.size_bytes,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\models.py:179:    size_bytes: int | None =
None
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\models.py:407:    size_bytes: int | None =
None
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\models.py:421:    """Composite key for
Tier-2 content dedup: (size_bytes, duration_s, height, width).
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\models.py:425:    if None in
(self.size_bytes, self.duration_s, self.height, self.width):
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\models.py:427:    return
(self.size_bytes, self.duration_s, self.height, self.width) # type: ignore[return-value]
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\db.py:77:    size_bytes INTEGER,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\db.py:123:    size_bytes          INTEGER,

```

```

C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\db.py:272: size_bytes: int | None =
None,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\db.py:302: if None not in
(size_bytes, duration_s, height, width):
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\db.py:303: return
f"tg:fp:{size_bytes}:{duration_s}:{height}:{width}"
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\db.py:464: size_bytes
INTEGER,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\db.py:473: "height, size_bytes,
mime_type, matched, forwarded, seen_at",
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\db.py:565: document_id,
duration_s, width, height, size_bytes, mime_type,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\db.py:574: v.duration_s,
v.width, v.height, v.size_bytes, v.mime_type,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\db.py:604: 'tg:fp:' ||
size_bytes || ':' || duration_s || ':'
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\db.py:607: AND size_bytes IS NOT
NULL AND duration_s IS NOT NULL
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\db.py:613: AND size_bytes IS
NOT NULL AND duration_s IS NOT NULL
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\db.py:616: GROUP BY account_id,
size_bytes, duration_s, height, width
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\db.py:876: size_bytes=row["size_bytes"],
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\db.py:1184: size_bytes,
mime_type, dedup_key, matched, seen_at)
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\db.py:1199: item.size_bytes,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\db.py:1536: size_bytes: int,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\db.py:1561: AND ci.size_bytes =
?
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\db.py:1566: (user_id, size_bytes,
duration_s, height, width),
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\db.py:1585: size_bytes=row["size_bytes"],
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\db.py:1686: size_bytes=video.size_bytes,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\db.py:1693: size_bytes=video.size_bytes,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\db.py:1744: size_bytes=item.size_bytes,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\db.py:1828: size_bytes=r["size_bytes"],
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\delivery.py:51: def _format_size(size_bytes:
int | None) -> str:
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\delivery.py:53: if size_bytes is None:
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\delivery.py:55: mib = size_bytes / (1024
* 1024)
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\delivery.py:73: size_bytes: int | None,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\delivery.py:88: f"?
{_format_height(height)} · {_format_duration(duration_s)} · {_format_size(size_bytes)}"
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\policy.py:737: size_bytes =
ctx.content_item.size_bytes
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\policy.py:738: if size_bytes is None:
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\policy.py:742: missing_size_bytes=True,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\policy.py:745: if size_bytes >
budget.max_bytes_per_item:
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\policy.py:751: size_bytes=size_bytes,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\filters\size.py:14: - If
`VideoMeta.size_bytes` is `None` (unknown), the rule fails.
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\filters\size.py:23: if
meta.size_bytes is None:
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\filters\size.py:26: if
meta.size_bytes > max_bytes:

```

```

C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\filters\size.py:27:         mib =
meta.size_bytes / _MIB
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\filters\size.py:29:         return
FilterResult.pass_(f"size {meta.size_bytes}B <= max {self.max_size_mb}MiB")
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\filters\base.py:26:         size_bytes: int |
None = None

```

8. assistant (2026-07-06T23:20:08.971Z)

Now let me run the repro to confirm the behavior described.

9. user (2026-07-06T23:20:10.233Z)

```

560             metadata,
561         )
562         return StageResult.pass_(
563             self.stage_name,
564             "caption safety has no valid active checks",
565             **metadata,
566         )
567         return StageResult.pass_(self.stage_name, "caption safety disabled")
568
569     if config.has_config_warnings:
570         log.warning(
571             "caption safety ignored invalid or excess config policy_id=%s "
572             "invalid_patterns=%s ignored_terms=%s ignored_patterns=%s",
573             ctx.policy.id,
574             list(config.invalid_patterns),
575             config.ignored_term_count,
576             config.ignored_pattern_count,
577         )
578
579     caption = (ctx.content_item.text or "").strip()
580     if not caption:
581         return StageResult.pass_(
582             self.stage_name,
583             "no caption to scan",
584             **_caption_safety_metadata(config, caption_present=False),
585         )
586
587     scan_caption = caption[:_MAX_CAPTION_SCAN_CHARS]
588     caption_truncated = len(caption) > _MAX_CAPTION_SCAN_CHARS
589     haystack = scan_caption.casefold()
590     for term in config.unsafe_terms:
591         if term.casefold() in haystack:
592             return StageResult.fail(
593                 self.stage_name,
594                 f"unsafe caption term matched: {term}",
595                 **_caption_safety_metadata(
596                     config,
597                     caption_present=True,
598                     caption_truncated=caption_truncated,
599                 ),
600                 match_type="term",
601                 matched_caption_term=term,
602             )
603
604     timed_out_patterns: list[str] = []
605     for pattern in config.unsafe_patterns:
606         try:
607             matched_pattern = regex.search(
608                 pattern,
609                 scan_caption,
610                 flags=regex.IGNORECASE,
611                 timeout=_CAPTION_REGEX_TIMEOUT_SECONDS,

```

```

612         )
613     except TimeoutError:
614         timed_out_patterns.append(pattern)
615         log.warning(
616             "caption safety regex timed out policy_id=%s pattern=%r",
617             ctx.policy.id,
618             pattern,
619         )
620         continue
621
622     if matched_pattern:
623         return StageResult.fail(
624             self.stage_name,
625             "unsafe caption pattern matched",
626             **_caption_safety_metadata(
627                 config,
628                 caption_present=True,
629                 caption_truncated=caption_truncated,

```

10. assistant (2026-07-06T23:20:12.356Z)

Now let me look at PolicyContext and PolicyBudget to write a correct repro.

11. user (2026-07-06T23:20:14.524Z)

```

C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\models.py:149:class ContentItem(BaseModel):
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\telethon_client.py:435:    content_item:
ContentItem,
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\policy.py:44:class PolicyBudget:
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\policy.py:61:    async def inspect(self,
fetcher: ContentFetcher, content_item: ContentItem) -> StageResult:
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\policy.py:75:    async def inspect(self,
content_item: ContentItem, budget: PolicyBudget) -> StageResult:
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\policy.py:81:class PolicyContext:
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\policy.py:84:    content_item: ContentItem
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\policy.py:87:    fetcher: ContentFetcher |
None = None
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\policy.py:88:    budget: PolicyBudget |
None = None
C:\Users\avidu\Projects\wt-pr54\src\tg_video_filter\policy.py:721:    budget = ctx.budget

```

12. user (2026-07-06T23:20:17.101Z)

```

40     _TRAILING_URL_PUNCTUATION = ". , ! ? ; : "
41
42
43     @dataclass(frozen=True)
44     class PolicyBudget:
45         """Bounded execution budget for opt-in deep inspection."""
46
47         max_bytes_per_item: int
48         timeout_s: float
49         max_concurrent: int
50         _semaphore: asyncio.Semaphore | None = field(
51             default=None,
52             init=False,
53             repr=False,
54             compare=False,
55         )
56
57         def __post_init__(self) -> None:
58             if self.max_concurrent > 0:
59                 object.__setattr__(self, "_semaphore",
asyncio.Semaphore(self.max_concurrent))

```

```

60
61     async def inspect(self, fetcher: ContentFetcher, content_item: ContentItem) ->
StageResult:
62         """Run ``fetcher`` under the timeout and concurrency budget."""
63         if self._semaphore is None:
64             raise ValueError("max_concurrent must be positive")
65         async with self._semaphore:
66             return await asyncio.wait_for(
67                 fetcher.inspect(content_item, self),
68                 timeout=self.timeout_s,
69             )
70
71
72     class ContentFetcher(Protocol):
73         """Fetcher used only by opt-in Tier 4 policy stages."""
74
75         async def inspect(self, content_item: ContentItem, budget: PolicyBudget) ->
StageResult:
76             """Inspect content under ``budget`` and return a policy result."""
77             ...
78
79
80     @dataclass(frozen=True)
81     class PolicyContext:
82         """Immutable snapshot available to a policy stage."""
83
84         content_item: ContentItem
85         source: Source
86         policy: Policy
87         fetcher: ContentFetcher | None = None
88         budget: PolicyBudget | None = None
89

```

13. assistant (2026-07-06T23:20:19.871Z)

Now let me write the repro.

14. assistant (2026-07-06T23:20:25.764Z)

```

import asyncio
from tg_video_filter.policy import (
    PolicyBudget, PolicyContext, DeepInspectionStage, _deep_inspection_budget_result,
)
from tg_video_filter.models import ContentItem, Policy, Source

class Fetcher:
    def __init__(self):
        self.called = False
    async def inspect(self, content_item, budget):
        self.called = True
        from tg_video_filter.policy import StageResult
        return StageResult.pass_("deep-inspection", "delegated")

def make_ctx(size, enabled=True):
    item = ContentItem(size_bytes=size)
    pol = Policy(id="p1", chat_id=1, deep_inspect=enabled) if False else None
    return item

async def main():
    # Build a minimal policy with deep_inspect enabled
    fields = {}
    # inspect ContentItem + Policy signatures

```



```
print("ContentItem fields:", ContentItem.model_fields.keys())
print("Policy fields:", list(Policy.model_fields.keys()))
print("Source:", list(Source))
```

```
asyncio.run(main())
```

15. user (2026-07-06T23:20:25.765Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

16. assistant (2026-07-06T23:20:27.685Z)

Let me check the model signatures first.

17. user (2026-07-06T23:20:28.625Z)

```
149     class ContentItem(BaseModel):
150         """A normalized content record discovered from any :class:`Source` (ADR 0011).
151
152         Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
153         (`content_type='video'`); `text` / `link_url` are landed now but stay
154         NULL until later tracks populate them (avoids a future `ALTER TABLE`).
155         `dedup_key` is the provider-neutral dedup key (Option 1A: tiered lookup
156         wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
157         """
158
159         model_config = ConfigDict(frozen=True)
160
161         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
162         account_id: int = Field(description="FK ? accounts.id")
163         source_id: int = Field(description="FK ? sources.id")
164         provider: str = Field(default="telegram", description="Provider namespace")
165         provider_message_id: str = Field(
166             description="Provider-native message id (Telegram message_id, ?)"
167         )
168         content_type: str = Field(default="video", description="'video' | 'link' |
169 'article'")
170         text: str | None = Field(default=None, description="Caption / post text (nullable in
171 Phase 1)")
172         link_url: str | None = Field(default=None, description="For link/article posts
173 (Phase 1: NULL)")
174         media_ref: str | None = Field(default=None, description="Provider media reference")
175         document_id: str | None = Field(
176             default=None,
177             description="Telegram document.id (kept for TG-specific Tier-1 lookups)",
178         )
179         duration_s: int | None = None
180         width: int | None = None
181         height: int | None = None
182         size_bytes: int | None = None
183         mime_type: str | None = None
184         dedup_key: str = Field(description="Provider-neutral dedup key (Option 1A); unique
185 per account")
186         matched: bool = False
187         seen_at: datetime = Field(default_factory=_utcnow)
188
189     class Policy(BaseModel):
190         """A named, reusable filtering policy owned by an account (ADR 0011, Option 2A).
191
192         Generalizes the legacy per-account `filter_config` row so one account can
193         own multiple policies (enables ADR 0009's per-route filter overrides) and so
194         future enrichment/security stages grow into the same `config_json` column.
195         Exactly one policy per account carries `is_default=True`; the legacy
```

```
193         ``load_filter_config`` / ``save_filter_config`` shims read/write that row.
```

18. assistant (2026-07-06T23:20:31.318Z)

Let me look at the Policy model and how deep_inspect config is loaded
(load_deep_inspection_enabled reads from policy).

19. user (2026-07-06T23:20:32.327Z)

```
186     class Policy(BaseModel):
187         """A named, reusable filtering policy owned by an account (ADR 0011, Option 2A).
188
189         Generalizes the legacy per-account ``filter_config`` row so one account can
190         own multiple policies (enables ADR 0009's per-route filter overrides) and so
191         future enrichment/security stages grow into the same ``config_json`` column.
192         Exactly one policy per account carries ``is_default=True``; the legacy
193         ``load_filter_config`` / ``save_filter_config`` shims read/write that row.
194         """
195
196         model_config = ConfigDict(frozen=True)
197
198         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
199         account_id: int = Field(description="FK ? accounts.id")
200         name: str = Field(description="Policy name; 'default' for the account default
policy")
201         config_json: str = Field(
202             description="FilterConfig JSON blob (Phase-1 shape); grows in Phase 2 (Track F)"
203         )
204         is_default: bool = Field(
205             default=False, description="Exactly one per account (UNIQUE partial index)"
206         )
207         created_at: datetime = Field(default_factory=_utcnow)
208         updated_at: datetime = Field(default_factory=_utcnow)
209
210
211         # --- Track C: destinations, routing, delivery state (ADR 0011) -----
212         #
213         # These tables were created empty by Track B (_SCHEMA); Track C populates
214         # destinations (backfilled from filter_config_deprecated.config_json), writes
215         # delivery_attempts (replacing the legacy videos.forwarded boolean), and
216         # defines the models for per-destination delivery state. The source/destination
217         # invariant (a channel is either source or destination, never both) is enforced
218         # at the app level by future selective-scanning logic (Track E, #30).
219
220
221     class Destination(BaseModel):
222         """A delivery target ? where matched content is sent (ADR 0011, Track C).
223
224         Generalizes the legacy ``FilterConfig.delivery_target`` field so one account
225         can have multiple named output channels. Backfilled from the existing
```

20. assistant (2026-07-06T23:20:34.763Z)

I'll test ``\_deep\_inspection\_budget\_result`` directly since it's the cited function. I need a
valid ContentItem and a PolicyContext.

21. assistant (2026-07-06T23:20:43.504Z)

```
import asyncio
from tg_video_filter.policy import (
    PolicyBudget, PolicyContext, DeepInspectionStage,
    _deep_inspection_budget_result, StageResult,
)
from tg_video_filter.models import ContentItem, Policy, Source
```

```

class Fetcher:
    def __init__(self):
        self.called = False

    async def inspect(self, content_item, budget):
        self.called = True
        return StageResult.pass_("deep-inspection", "delegated")

def make_item(size):
    return ContentItem(
        account_id=1,
        source_id=1,
        provider_message_id="m1",
        size_bytes=size,
        dedup_key="k1",
    )

def make_policy():
    return Policy(
        account_id=1,
        name="default",
        config_json='{"deep_inspect": true}',
        is_default=True,
    )

async def main():
    print("Source members:", list(Source))
    budget = PolicyBudget(max_bytes_per_item=100, timeout_s=5.0, max_concurrent=2)

    for size in (-5, 0, 50, 200, None):
        fetcher = Fetcher()
        ctx = PolicyContext(
            content_item=make_item(size),
            source=list(Source)[0],
            policy=make_policy(),
            fetcher=fetcher,
            budget=budget,
        )
        # direct budget-result check
        bres = _deep_inspection_budget_result(ctx)
        # full stage evaluate
        stage = DeepInspectionStage()
        res = await stage.evaluate(ctx)
        print(f"size={size!r}>6} budget_result={'None' if bres is None else
'FAIL:'+bres.reason:45} stage.passed={res.passed} reason={res.reason!r}
fetcher.called={fetcher.called}")

    asyncio.run(main())

```

22. user (2026-07-06T23:20:43.509Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

23. assistant (2026-07-06T23:20:45.854Z)

The file exists from a prior failed write attempt? No?the error is the tool guard. Let me use a different filename.

24. assistant (2026-07-06T23:20:51.922Z)

```

import asyncio
from tg_video_filter.policy import (
    PolicyBudget, PolicyContext, DeepInspectionStage,
    _deep_inspection_budget_result, StageResult,
)
from tg_video_filter.models import ContentItem, Policy, Source

class Fetcher:
    def __init__(self):
        self.called = False

    async def inspect(self, content_item, budget):
        self.called = True
        return StageResult.pass_("deep-inspection", "delegated")

def make_item(size):
    return ContentItem(
        account_id=1,
        source_id=1,
        provider_message_id="m1",
        size_bytes=size,
        dedup_key="k1",
    )

def make_policy():
    return Policy(
        account_id=1,
        name="default",
        config_json='{"deep_inspect": true}',
        is_default=True,
    )

async def main():
    print("Source members:", list(Source))
    budget = PolicyBudget(max_bytes_per_item=100, timeout_s=5.0, max_concurrent=2)

    for size in (-5, 0, 50, 200, None):
        fetcher = Fetcher()
        ctx = PolicyContext(
            content_item=make_item(size),
            source=list(Source)[0],
            policy=make_policy(),
            fetcher=fetcher,
            budget=budget,
        )
        bres = _deep_inspection_budget_result(ctx)
        stage = DeepInspectionStage()
        res = await stage.evaluate(ctx)
        b = "None" if bres is None else "FAIL:" + bres.reason
        print(f"size={size!r}>6} budget_result={b:60} stage.passed={res.passed}")
    reason={res.reason!r} fetcher.called={fetcher.called}")

asyncio.run(main())

```

25. user (2026-07-06T23:20:51.953Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/repro_size.py (file state is current in your context ? no need to Read it back)

26. assistant (2026-07-06T23:20:54.886Z)

Let me check how `load_deep_inspection_enabled` reads the policy ? it may read `config_json` vs a `deep_inspect` field.

27. user (2026-07-06T23:20:56.582Z)

```
294: def load_deep_inspection_enabled(policy: Policy) -> bool:
295-     """Return whether Tier 4 deep inspection is explicitly enabled."""
296-     decoded = json.loads(policy.config_json)
297-     if not isinstance(decoded, dict):
298-         raise ValueError("policy config_json must decode to a JSON object")
299-     raw_deep_inspect = decoded.get("deep_inspect", False)
300-     if isinstance(raw_deep_inspect, bool):
301-         return raw_deep_inspect
302-     if isinstance(raw_deep_inspect, dict):
303-         enabled = raw_deep_inspect.get("enabled", False)
304-         if isinstance(enabled, bool):
305-             return enabled
306-     raise ValueError("deep_inspect must be a boolean or an object with boolean enabled")
307-
308-
309- def _string_tuple(value: object, *, field_name: str) -> tuple[str, ...]:
310-     if value is None:
311-         return ()
312-     if not isinstance(value, list):
313-         raise ValueError(f"{field_name} must be an array")
314-     values: list[str] = []
315-     for item in value:
316-         if not isinstance(item, str):
317-             raise ValueError(f"{field_name} must contain strings")
318-         text = item.strip()
319-         if text:
320-             values.append(text)
321-     return tuple(values)
322-
323-
324- class TopicPreferenceStage:
325-     """ADR 0012 Tier 2 topic matching over source title and captured caption."""
326-
327-     stage_name = "topic-preferences"
328-
329-     async def evaluate(self, ctx: PolicyContext) -> StageResult:
330-         try:
331-             prefs = load_topic_preferences(ctx.policy)
332-             --
333-             enabled = load_deep_inspection_enabled(ctx.policy)
334-         except (json.JSONDecodeError, ValueError) as exc:
335-             log.warning(
336-                 "deep inspection disabled due to invalid config policy_id=%s",
337-                 ctx.policy.id,
338-                 exc_info=True,
339-             )
340-             return StageResult.pass_(
341-                 self.stage_name,
342-                 f"invalid deep inspection config ignored: {exc}",
343-                 invalid_deep_inspection_config=True,
344-             )
345-         if not enabled:
346-             return StageResult.pass_(self.stage_name, "deep inspection disabled")
347-
348-         budget_result = _deep_inspection_budget_result(ctx)
349-         if budget_result is not None:
350-             return budget_result
```

```

700-         if ctx.fetcher is None:
701-             return StageResult.fail(
702-                 self.stage_name,
703-                 "deep inspection fetcher missing",
704-                 missing_fetcher=True,
705-             )
706-
707-         assert ctx.budget is not None
708-         try:
709-             return await ctx.budget.inspect(ctx.fetcher, ctx.content_item)
710-         except TimeoutError:
711-             return StageResult.fail(
712-                 self.stage_name,
713-                 "deep inspection budget exceeded",
714-                 budget_exceeded=True,
715-                 exceeded_budget="timeout_s",
716-                 timeout_s=ctx.budget.timeout_s,
717-             )
718-
719-
720-def _deep_inspection_budget_result(ctx: PolicyContext) -> StageResult | None:
721-    budget = ctx.budget
722-    if budget is None:
723-        return StageResult.fail(
724-            DeepInspectionStage.stage_name,
725-            "deep inspection budget missing",
726-            missing_budget=True,
727-        )
728-    if budget.max_bytes_per_item <= 0 or budget.timeout_s <= 0 or budget.max_concurrent <=
0:
729-        return StageResult.fail(
730-            DeepInspectionStage.stage_name,
731-            "deep inspection budget invalid",
732-            invalid_budget=True,
733-            max_bytes_per_item=budget.max_bytes_per_item,
734-            timeout_s=budget.timeout_s,
735-            max_concurrent=budget.max_concurrent,
736-        )
737-    size_bytes = ctx.content_item.size_bytes
738-    if size_bytes is None:
739-        return StageResult.fail(
740-            DeepInspectionStage.stage_name,
741-            "deep inspection budget cannot be enforced without content size",
742-            missing_size_bytes=True,
743-            max_bytes_per_item=budget.max_bytes_per_item,
744-        )
745-    if size_bytes > budget.max_bytes_per_item:
746-        --
788:    "load_deep_inspection_enabled",
789:    "load_domain_reputation_config",
790:    "load_topic_preferences",
791:    "normalize_domain",
792:    "parse_caption_safety_config",
793:    "parse_domain_reputation_config",
794:    "parse_topic_preferences",
795-]

```

28. assistant (2026-07-06T23:20:59.425Z)

My repro uses `config\_json={'deep\_inspect': true}` which is correct. Let me run it.

29. user (2026-07-06T23:21:03.349Z)

Exit code 1

Traceback (most recent call last):

```
size=-5 budget_result=None
stage.passed=True reason='delegated'  fetcher.called=True
size= 0 budget_result=None
stage.passed=True reason='delegated'  fetcher.called=True
size= 50 budget_result=None
stage.passed=True reason='delegated'  fetcher.called=True
size= 200 budget_result=FAIL:deep inspection budget exceeded
stage.passed=False reason='deep inspection budget exceeded' fetcher.called=False
```

```
size= None budget_result=FAIL:deep inspection budget cannot be enforced without content size
stage.passed=False reason='deep inspection budget cannot be enforced without content size'
fetcher.called=False
```

34. assistant (2026-07-06T23:21:32.248Z)

The repro confirms the described behavior exactly: ``size_bytes=-5`` and ``size_bytes=0`` both yield ``budget_result=None`` (in-budget) and the fetcher is called, whereas ``None`` is rejected and ``200`` (`>max`) is rejected.

Now the adversarial assessment. The finding claims the defect ships in this PR's diff and produces the described behavior. That is factually confirmed. The question is severity and whether it's actually a meaningful defect.

Key considerations for refutation:

1. ****Can a negative size_bytes actually occur?**** `size_bytes` flows from ``media.size`` (telethon_client.py:97). Telegram document sizes are non-negative. The finding's premise is "buggy/hostile provider metadata path." This is speculative ? there's no demonstrated path producing a negative size. But `size_bytes=0` (empty/zero-byte item) is entirely plausible and also delegates.
2. ****Is delegating a small/zero/negative-size item to the fetcher harmful?**** The finding itself concedes: the delegated work is still bounded by timeout + semaphore. A size of -5 or 0 is **smaller** than the budget, so from a budget-enforcement standpoint it is not a bypass of the byte cap ? heavy work isn't running "unbounded." The byte budget's purpose is to reject items **too large** to inspect cheaply; a tiny/zero/negative size is within the "cheap to inspect" regime the budget is designed to permit.
3. ****Is this even wrong behavior?**** The missing-size branch fails because size is **unknown** ? you can't verify it fits. A negative size is arguably "known and nonsensical," but the safe-direction argument (refuse untrustworthy metadata) is a defense-in-depth preference, not a correctness bug that produces wrong output. The fetcher receives the item and does its own inspection; nothing in the PR's contract says the fetcher assumes a positive size.

So: the code behavior is exactly as described (confirmed factually), but it is not a budget/timeout/concurrency bypass ? the delegated work remains bounded. This is a low-severity defense-in-depth / correctness nit, exactly as the finding self-rates. It genuinely ships in the diff. Severity low is correct.

Verdict: CONFIRMED at low severity. The behavior is real and in-diff, but it is a hardening nit, not a bypass ? consistent with the finding's own "low" rating.

35. user (2026-07-06T23:21:42.675Z)

Structured output provided successfully